EVOLUTIONARY COMPUTATION — EXERCISE 3

Local Search for the Node-Selection TSP

Steepest / greedy · two-nodes / two-edges exchange · random / greedy starting solutions

Instances A and B · 200 nodes each · 100 selected per tour
Report generated 2026-11-02

Wiktor Bocheński · Mikołaj Dybizbański

1. Description of the problem

We are given two problem instances, **A** and **B**, each listing 200 nodes. Every node is described by three integers: its `x` and `y` coordinates on a plane, and a `cost`. Node indices are 0-based, taken from the row order in the instance file.

Goal. Select exactly 50% of the nodes (rounded up if the count is odd — for 200 nodes, exactly 100) and arrange the selected nodes into a **Hamiltonian cycle**: a closed path that visits every selected node exactly once and returns to its start. The objective to minimize is:

```
objective = total length of the cycle + sum of costs of the selected nodes
```

Distances between nodes are **Euclidean**, computed from the `(x, y)` coordinates and **rounded mathematically** (round-half-away-from-zero) to the nearest integer. The distance matrix is built once, immediately after reading an instance; from that point on, every optimization method only ever reads the distance matrix and the cost list — never the raw coordinates — so the same methods would work unchanged for an instance given purely as a distance matrix, with no coordinates at all.

A solution is a sequence of 100 distinct node indices, in visiting order; the cycle is implied by wrapping the last node back to the first. A solution is valid iff it has exactly `ceil(n/2)` nodes, every index is in range, and no index repeats.

2. Implemented methods

Neighborhood

The neighborhood of a solution is the union of two kinds of moves, always evaluated together:

- **Inter-route move** — exchange of a selected node (at tour position *i*) with an unselected node: the new node takes the old node's place in the cycle. 100 × 100 = 10 000 moves.
- **Intra-route move** (changes only the order of the selected nodes), one of two options:
  - **two-nodes exchange** — swap the nodes at tour positions $i < j$ (4 950 moves);
  - **two-edges exchange** (2-opt) — remove edges (t_i, t_{i+1}) and (t_j, t_{j+1}), add (t_i, t_j) and (t_{i+1}, t_{j+1}) by reversing the segment $t_{i+1..j}$ (4 850 moves; pairs of adjacent edges are skipped since they give the identity).

Every move is evaluated by its **delta** of the objective function, computed only from the 2–4 edges and node costs it touches (formulas in section 3); the neighbor solution is never built and never evaluated from scratch. Moves are identified by tour positions (and by a position in the list of unselected nodes), so the list of all moves is built once and stays valid while the search runs. Only the chosen move is applied to the solution.

Type of local search

- **Steepest** — evaluate the whole neighborhood (both move kinds), apply the best move if it improves the objective; stop when no move improves.
- **Greedy** — browse the neighborhood in random order and apply the *first* improving move; stop when a full pass finds no improving move.

Randomization of the greedy version. All moves of both kinds (intra- and inter-route) are kept in one list, and the order is generated *lazily* with an incremental Fisher–Yates shuffle. To pick the next move, draw a random index among the moves not yet tried in this step, swap it to the front of the untried part, and evaluate only that move. This gives a uniformly random permutation of the whole neighborhood: both move kinds are mixed, and no kind is preferred or always tried first. Each random draw costs O(1), so the search never evaluates (or even touches) moves past the first improving one. After a move is applied, the next step simply keeps drawing from the same list. Any prefix of a Fisher–Yates shuffle is a uniform random sample, so no reset is needed.

Starting solutions

- **Random** — 200 random solutions (100 random distinct nodes in random order). The *same* 200 solutions are used by all four random-start variants.
- **Greedy** — the best constructive heuristic so far (best average objective among all previous methods): **Weighted 2-regret + delta (greedy cycle)** on instance A and **Weighted 2-regret + delta (NN, any position)** on instance B (see exercise 2), started from each of the 200 nodes.

Three binary options (steepest/greedy × nodes/edges exchange × random/greedy start) give 8 methods, each run 200 times per instance. All previous methods were re-run in the same program, so every time below has the same basis. Times are per run and include building the starting solution.

3. Pseudocode

3.1 Move deltas (t = tour, k = 100, indices mod k, d = distances, c = costs)

```
interDelta(i, w):            # selected v = t[i] replaced by unselected w
    p, n = t[i-1], t[i+1]
    return d[p][w] + d[w][n] - d[p][v] - d[v][n] + c[w] - c[v]

nodesExchangeDelta(i, j):    # i < j, a = t[i], b = t[j]
    if j == i+1:             # ... p a b n ...  ->  ... p b a n ...
        return d[t[i-1]][b] + d[a][t[j+1]] - d[t[i-1]][a] - d[b][t[j+1]]
    if i == 0 and j == k-1:  # adjacent through the wrap-around edge
        return d[t[j-1]][a] + d[b][t[i+1]] - d[t[j-1]][b] - d[a][t[i+1]]
    return d[t[i-1]][b] + d[b][t[i+1]] + d[t[j-1]][a] + d[a][t[j+1]]
         - d[t[i-1]][a] - d[a][t[i+1]] - d[t[j-1]][b] - d[b][t[j+1]]

edgesExchangeDelta(i, j):    # i < j, edges not adjacent
    return d[t[i]][t[j]] + d[t[i+1]][t[j+1]] - d[t[i]][t[i+1]] - d[t[j]][t[j+1]]

apply(move):
    inter:  swap(t[i], unselected[u])
    nodes:  swap(t[i], t[j])
    edges:  reverse(t[i+1 .. j])
```

3.2 Steepest local search

```
steepestLocalSearch(start):
    t = start;  unselected = all nodes not in t
    moves = all intra moves (i, j) + all inter moves (i, u)     # built once
    repeat:
        best = None;  bestDelta = 0
        for m in moves:
            if delta(m) < bestDelta: best, bestDelta = m, delta(m)
        if best is None: return t                               # local optimum
        apply(best)
```

3.3 Greedy local search

```
greedyLocalSearch(start, rng):
    t = start;  unselected = all nodes not in t
    order = [0, 1, ..., |moves|-1]                              # built once
    repeat:
        improved = false
        for tried in 0 .. |order|-1:
            r = uniform random integer in [tried, |order|-1]
            swap(order[tried], order[r])                        # lazy Fisher-Yates
            m = moves[order[tried]]
            if delta(m) < 0:
                apply(m);  improved = true;  break              # first improvement
        if not improved: return t                               # local optimum
```

4. Computational experiment

Each method: 200 runs per instance (200 random starting solutions, or one run from each of the 200 starting nodes). Cells show **average (min – max)**; the best average per instance is shown in **green**.

Objective function

Method	Instance A	Instance B
Previous methods (constructive heuristics)		
Random solution	262519.4 (238710 – 294111)	207338.6 (180318 – 239655)
Nearest neighbor (end only)	87476.6 (84362 – 90075)	59397.3 (56007 – 63166)
Nearest neighbor (any position)	74347.0 (72328 – 77022)	59480.3 (52915 – 66164)
Greedy cycle	73797.4 (72590 – 75281)	59709.5 (53016 – 63736)
2-regret (NN, any position)	122860.7 (111527 – 133181)	74763.4 (69032 – 80767)
Weighted 2-regret + delta (NN, any position)	75561.2 (72419 – 78504)	55698.3 (50485 – 61406)
2-regret (greedy cycle)	121007.2 (112654 – 130924)	73403.4 (69455 – 78142)
Weighted 2-regret + delta (greedy cycle)	73348.7 (72011 – 75532)	57895.5 (52731 – 63370)
Local search (this exercise)		
Steepest LS, nodes exchange, random start	90051.5 (83416 – 97595)	66084.1 (58864 – 75837)
Steepest LS, nodes exchange, greedy start	73017.1 (71955 – 74682)	54292.1 (49978 – 61265)
Steepest LS, edges exchange, random start	76245.8 (73704 – 80426)	**52597.5 (50149 – 55884)**
Steepest LS, edges exchange, greedy start	72996.8 (71955 – 74682)	53091.3 (49346 – 60354)
Greedy LS, nodes exchange, random start	87787.9 (81532 – 98399)	63784.8 (58120 – 72965)
Greedy LS, nodes exchange, greedy start	73016.7 (71955 – 74682)	54298.5 (49913 – 61265)
Greedy LS, edges exchange, random start	76273.1 (74090 – 78637)	52935.7 (50579 – 55914)
Greedy LS, edges exchange, greedy start	**72979.8 (71955 – 74682)**	53094.7 (49727 – 60266)

Previous methods give the same values as in exercises 1–2 (deterministic). The only exception is the random solution on B, which uses a different part of the shared RNG stream.

Running time [ms per run]

Method	Instance A	Instance B
Previous methods (constructive heuristics)		
Random solution	0.006 (0.004 – 0.009)	0.005 (0.004 – 0.006)
Nearest neighbor (end only)	0.037 (0.032 – 0.043)	0.034 (0.032 – 0.050)
Nearest neighbor (any position)	0.886 (0.873 – 0.917)	0.861 (0.847 – 0.890)

Method	Instance A	Instance B
Greedy cycle	0.962 (0.951 – 0.999)	0.930 (0.911 – 0.962)
2-regret (NN, any position)	1.324 (1.301 – 1.356)	1.332 (1.312 – 1.365)
Weighted 2-regret + delta (NN, any position)	1.317 (1.298 – 1.355)	1.341 (1.319 – 1.515)
2-regret (greedy cycle)	1.541 (1.496 – 1.815)	1.545 (1.525 – 1.611)
Weighted 2-regret + delta (greedy cycle)	1.718 (1.492 – 3.830)	1.547 (1.523 – 1.611)
Local search (this exercise)		
Steepest LS, nodes exchange, random start	10.625 (8.395 – 16.335)	10.652 (7.682 – 16.370)
Steepest LS, nodes exchange, greedy start	1.853 (1.562 – 2.569)	2.152 (1.420 – 3.014)
Steepest LS, edges exchange, random start	7.617 (6.476 – 10.921)	7.497 (6.236 – 8.585)
Steepest LS, edges exchange, greedy start	1.774 (1.558 – 2.881)	2.189 (1.533 – 3.053)
Greedy LS, nodes exchange, random start	5.317 (2.794 – 9.953)	4.967 (2.625 – 10.444)
Greedy LS, nodes exchange, greedy start	2.860 (2.103 – 4.351)	3.429 (1.980 – 7.962)
Greedy LS, edges exchange, random start	3.557 (2.042 – 6.779)	3.462 (1.971 – 5.080)
Greedy LS, edges exchange, greedy start	2.913 (2.132 – 5.730)	3.752 (2.294 – 6.909)

Times include constructing the starting solution, so greedy-start rows contain ≈1.7 ms (A) / ≈1.3 ms (B) of construction. Measured on Apple Silicon, clang++ -O2, single thread.

Search effort (average per run)

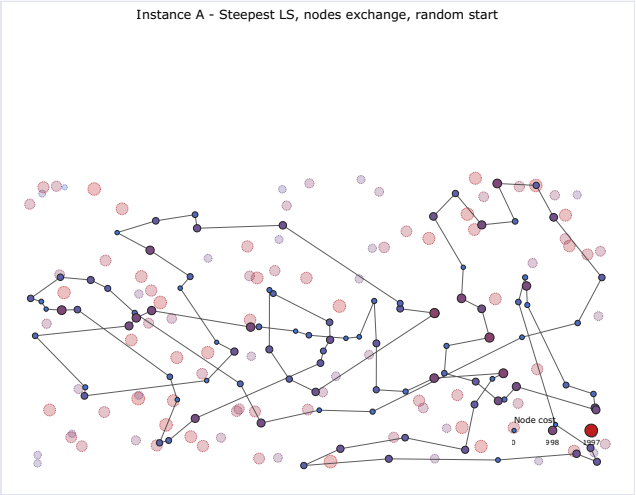
Method	A: evaluated moves	A: applied moves	B: evaluated moves	B: applied moves
Steepest LS, nodes exchange, random start	2,288,247	152.1	2,275,689	151.2
Steepest LS, nodes exchange, greedy start	64,136	3.3	171,476	10.5
Steepest LS, edges exchange, random start	1,989,752	133.0	1,981,584	132.4
Steepest LS, edges exchange, greedy start	63,781	3.3	227,056	14.3
Greedy LS, nodes exchange, random start	138,808	417.6	132,701	414.2
Greedy LS, nodes exchange, greedy start	32,823	4.2	52,046	15.2
Greedy LS, edges exchange, random start	96,012	419.3	95,364	421.3
Greedy LS, edges exchange, greedy start	33,418	4.6	63,522	26.7

Raw data: 3_local/results/A_summary.csv, B_summary.csv.

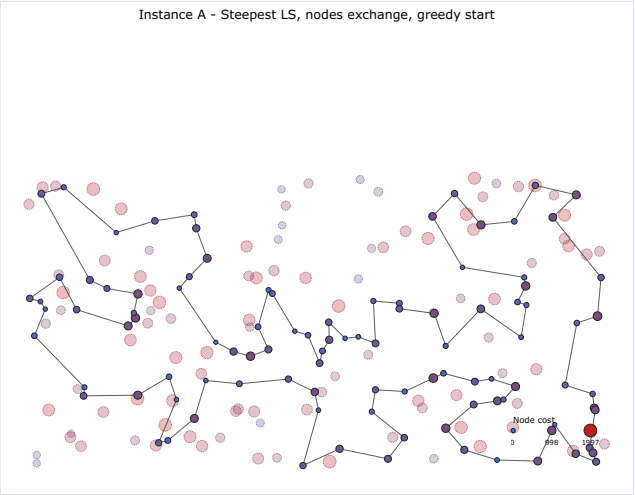
5. 2D visualizations of the best solution per instance/method

Node color and size encode cost (small/blue = cheap, large/red = expensive). Unselected nodes use the same cost encoding but are drawn translucent with a dashed outline; the closed tour is drawn as a dark path.

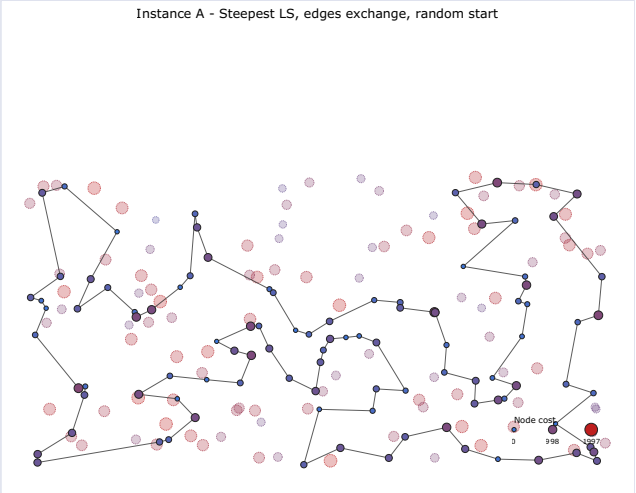
Instance A



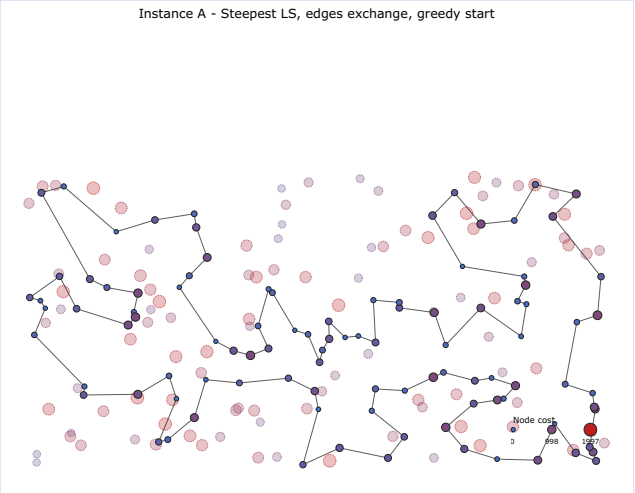
Steepest LS, nodes exchange, random start



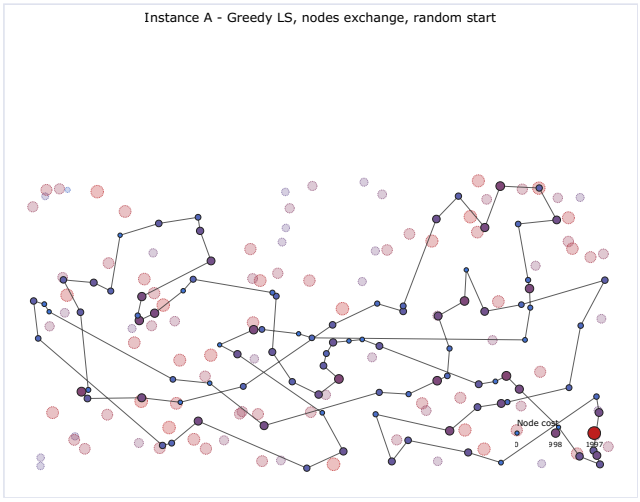
Steepest LS, nodes exchange, greedy start



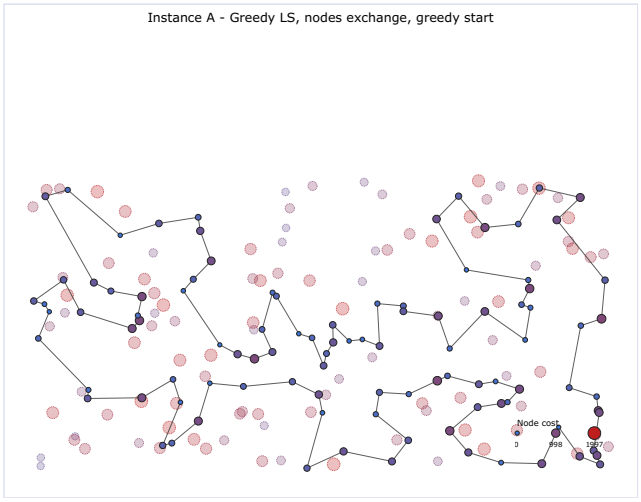
Steepest LS, edges exchange, random start



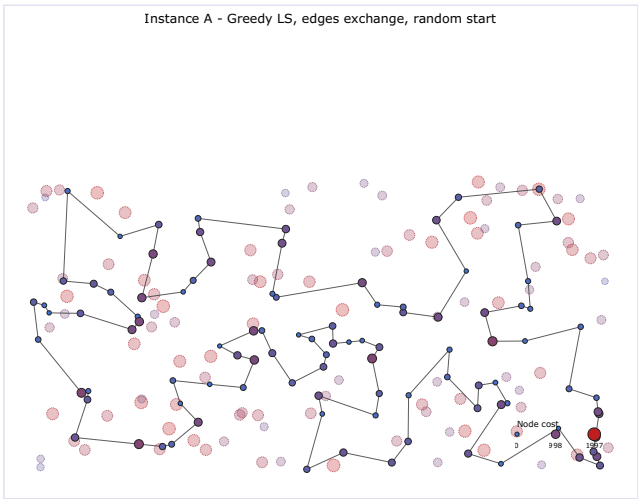
Steepest LS, edges exchange, greedy start



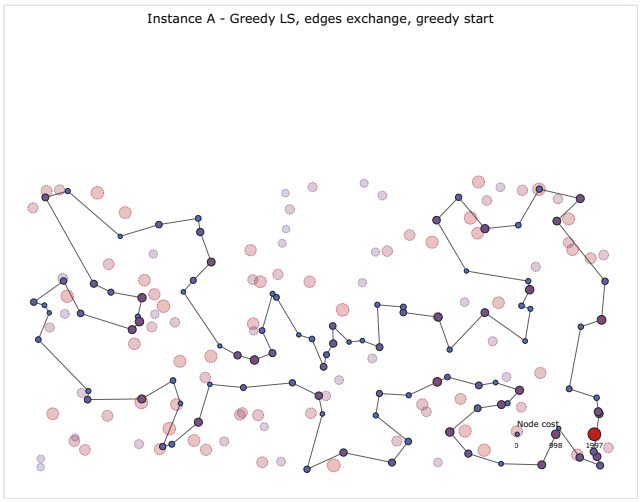
Greedy LS, nodes exchange, random start



Greedy LS, nodes exchange, greedy start

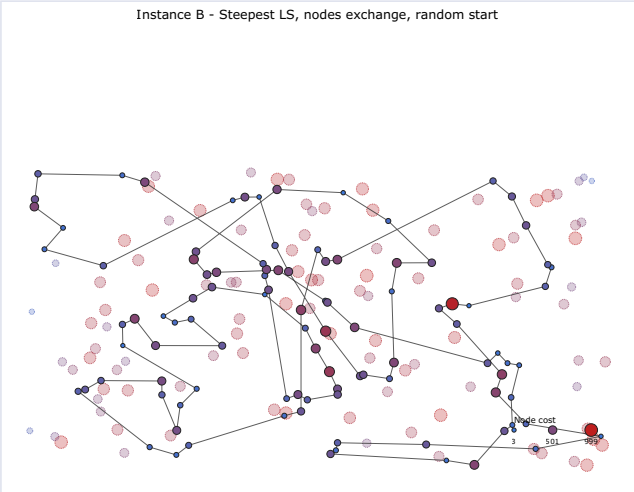


Greedy LS, edges exchange, random start

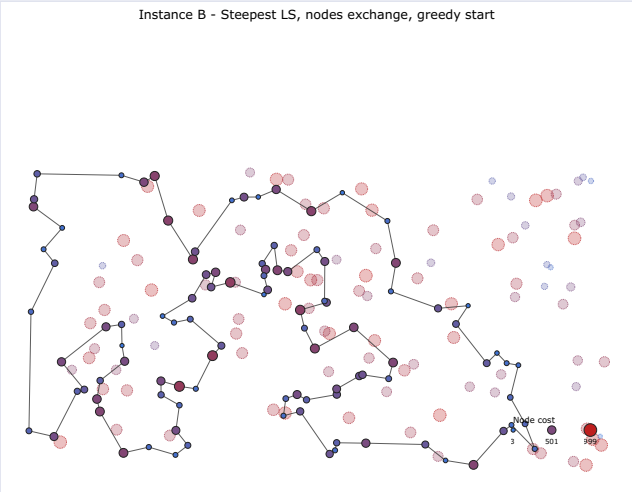


Greedy LS, edges exchange, greedy start

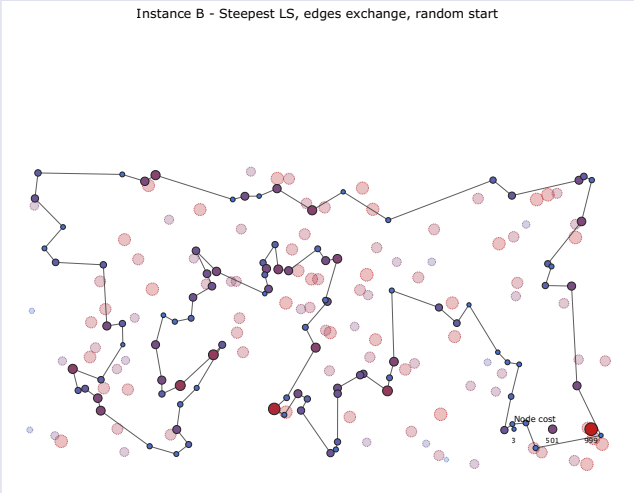
Instance B



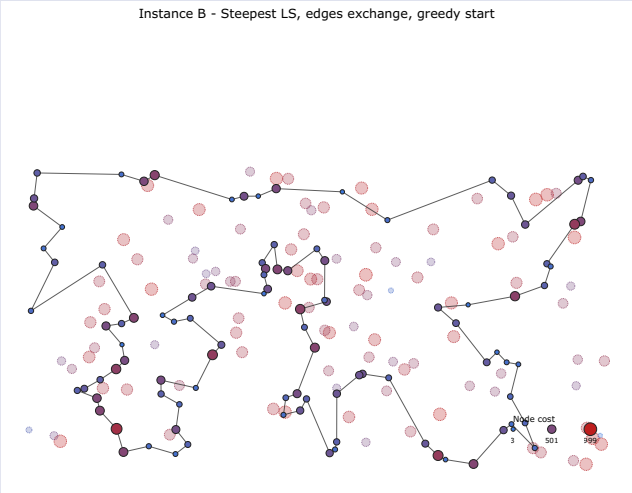
Steepest LS, nodes exchange, random start



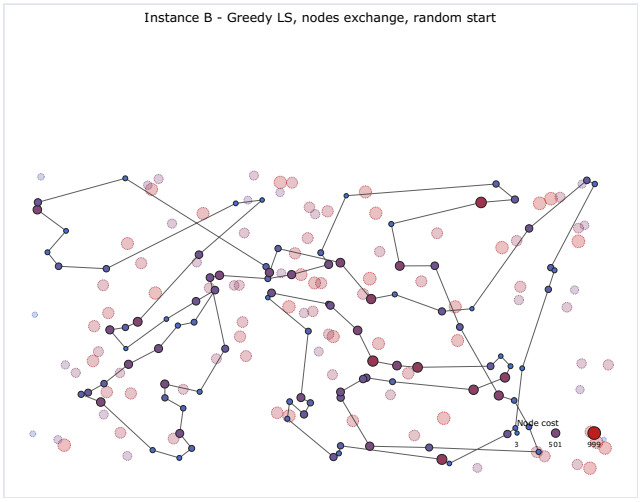
Steepest LS, nodes exchange, greedy start



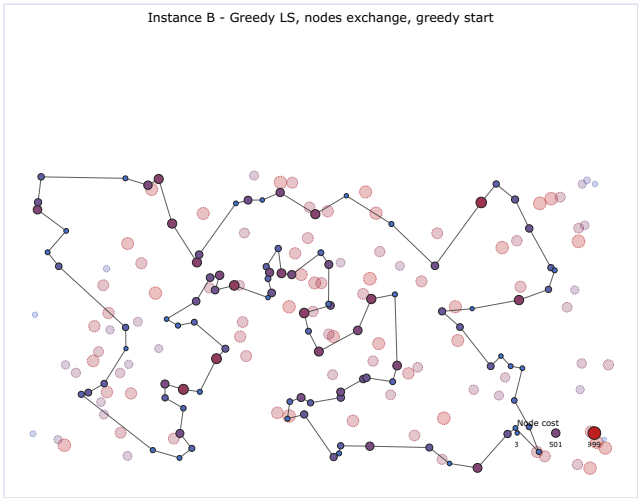
Steepest LS, edges exchange, random start



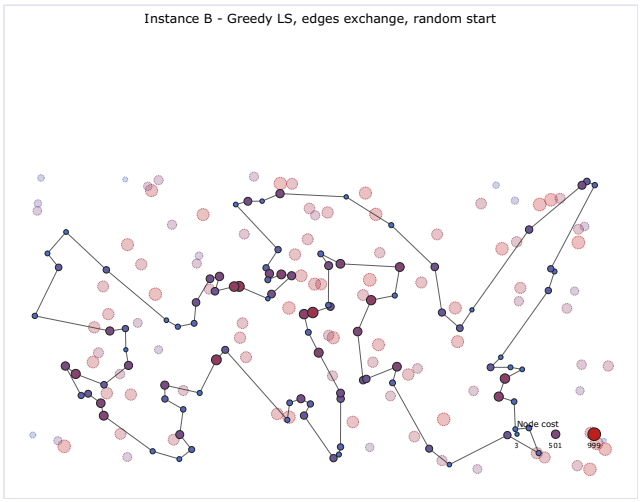
Steepest LS, edges exchange, greedy start



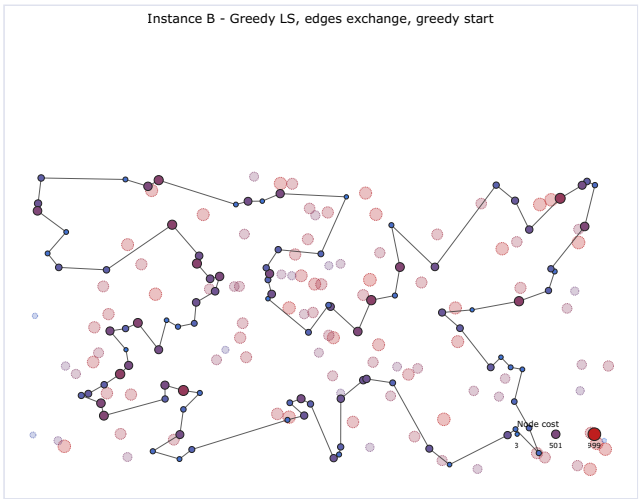
Greedy LS, nodes exchange, random start



Greedy LS, nodes exchange, greedy start



Greedy LS, edges exchange, random start



Greedy LS, edges exchange, greedy start

6. Best solutions (node indices, 0-based)

Instance A, Steepest LS, nodes exchange, random start `objective 83416`

144,177,36,33,131,84,134,72,43,106,151,182,158,181,114,192,57,88,78,153,107,3,1,169,50,130,159,146,170,113,89,42,150,176,174,81,85,142,164,186,136,184,178,163,29,101,147,47,93,58,135,98,87,123,152,108,60,116,149,63,11,49,30,187,171,2,189,140,53,179,52,16,51,185,45,73,8,194,54,9,112,71,157,91,80,38,76,132,180,28,141,127,97,168,160,83,90,156,35,67

Instance A, Steepest LS, nodes exchange, greedy start `objective 71955`

101,147,37,47,130,159,146,170,153,193,78,88,107,64,57,182,138,151,42,89,113,106,150,176,72,134,84,58,158,192,181,114,135,73,45,87,141,189,2,171,187,30,161,127,97,168,63,149,116,60,49,160,83,123,100,152,108,31,48,80,91,157,71,4,76,132,180,28,102,185,35,51,16,67,144,140,53,179,52,177,36,131,33,9,54,8,174,85,93,50,3,169,1,164,186,136,184,178,29,55

Instance A, Steepest LS, edges exchange, random start `objective 73704`

69,127,97,95,168,63,149,116,60,31,48,80,108,49,152,123,83,90,132,76,157,71,4,51,16,67,144,52,33,9,54,194,8,174,176,150,151,42,89,113,106,93,50,3,169,1,164,178,184,136,163,29,55,101,147,37,47,130,159,107,153,193,78,88,64,57,182,138,158,192,181,114,135,58,84,134,131,36,177,179,53,140,73,45,185,35,156,180,28,102,87,141,30,161,187,189,2,171,104,129

Instance A, Steepest LS, edges exchange, greedy start `objective 71955`

55,29,178,184,136,186,164,1,169,3,50,93,85,174,8,54,9,33,131,36,177,52,179,53,140,144,67,16,51,35,185,102,28,180,132,76,4,71,157,91,80,48,31,108,152,100,123,83,160,49,60,116,149,63,168,97,127,161,30,187,171,2,189,141,87,45,73,135,114,181,192,158,58,84,134,72,176,150,106,113,89,42,151,138,182,57,64,107,88,78,193,153,170,146,159,130,47,37,147,101

Instance A, Greedy LS, nodes exchange, random start `objective 81532`

152,108,60,49,97,95,127,161,187,87,52,33,54,9,186,136,184,163,29,101,178,1,169,50,93,144,67,35,156,180,28,135,181,114,189,2,171,168,116,149,63,30,141,98,84,134,72,176,174,8,142,164,85,3,147,47,130,89,42,151,138,58,192,158,182,57,159,170,153,193,78,88,107,113,43,106,150,131,36,177,179,53,140,41,73,45,185,16,51,76,132,90,83,123,100,4,71,157,91,80

Instance A, Greedy LS, nodes exchange, greedy start `objective 71955`

101,147,37,47,130,159,146,170,153,193,78,88,107,64,57,182,138,151,42,89,113,106,150,176,72,134,84,58,158,192,181,114,135,73,45,87,141,189,2,171,187,30,161,127,97,168,63,149,116,60,49,160,83,123,100,152,108,31,48,80,91,157,71,4,76,132,180,28,102,185,35,51,16,67,144,140,53,179,52,177,36,131,33,9,54,8,174,85,93,50,3,169,1,164,186,136,184,178,29,55

Instance A, Greedy LS, edges exchange, random start `objective 74090`

35,185,45,140,53,144,67,52,179,177,36,131,99,84,58,73,135,114,181,192,158,134,174,176,151,150,106,89,42,182,57,107,88,78,193,153,75,170,146,159,130,47,93,81,85,3,50,1,178,101,29,136,186,164,8,54,9,33,143,16,51,139,112,157,71,4,76,132,100,38,91,80,48,60,108,152,123,83,160,49,63,149,116,168,95,97,127,69,92,171,2,189,187,30,141,87,102,28,180,156

Instance A, Greedy LS, edges exchange, greedy start `objective 71955`

55,29,178,184,136,186,164,1,169,3,50,93,85,174,8,54,9,33,131,36,177,52,179,53,140,144,67,16,51,35,185,102,28,180,132,76,4,71,157,91,80,48,31,108,152,100,123,83,160,49,60,116,149,63,168,97,127,161,30,187,171,2,189,141,87,45,73,135,114,181,192,158,58,84,134,72,176,150,106,113,89,42,151,138,182,57,64,107,88,78,193,153,170,146,159,130,47,37,147,101

Instance B, Steepest LS, nodes exchange, random start `objective 58864`

```
134,175,132,29,6,155,71,157,60,40,170,125,26,97,64,47,140,154,38,21,69,151,98,25,152,181,167,73,59,160,172,136,133,
147,116,37,146,13,149,28,158,121,198,96,120,117,111,34,24,67,66,76,185,104,57,89,48,130,191,30,183,8,184,3,27,80,12
7,129,90,153,20,74,17,171,82,87,179,99,2,110,14,182,52,192,162,159,54,10,114,196,187,70,190,86,46,72,122,51,85,45
```

Instance B, Steepest LS, nodes exchange, greedy start `objective 49978`

```
65,145,28,149,13,146,37,116,147,42,50,191,130,121,198,96,48,156,89,57,185,76,162,54,192,80,127,129,90,153,14,2,20,7
4,17,171,99,88,179,87,82,51,85,59,73,167,181,152,34,24,67,66,27,98,151,122,3,184,46,72,117,8,120,133,136,172,160,69
,189,21,183,30,38,154,140,47,64,33,40,141,155,71,60,157,45,134,175,62,36,197,6,115,170,125,177,178,29,132,174,1
```

Instance B, Steepest LS, edges exchange, random start `objective 50149`

```
152,181,179,87,82,51,167,73,85,107,59,98,151,3,184,86,46,72,117,8,120,133,136,172,160,69,183,130,30,21,38,47,140,15
4,97,155,71,141,33,64,40,60,157,45,134,175,36,197,29,132,103,6,170,125,177,158,145,28,149,146,37,116,147,42,121,198
,96,48,156,89,57,190,70,81,94,105,61,196,114,10,148,118,110,2,14,20,74,153,90,129,127,54,192,162,76,66,67,31,24,34
```

Instance B, Steepest LS, edges exchange, greedy start `objective 49346`

```
65,145,28,149,13,146,37,116,147,42,121,198,96,48,89,57,190,70,187,81,94,105,61,139,196,114,10,78,54,162,192,80,127,
129,90,153,14,2,20,74,17,171,39,99,67,24,34,181,179,87,82,167,51,85,59,73,98,151,122,3,184,46,72,117,8,120,133,136,
172,160,69,21,38,154,140,47,64,33,40,155,71,60,157,45,134,175,62,91,36,197,132,29,6,180,115,170,177,125,26,158
```

Instance B, Greedy LS, nodes exchange, random start `objective 58120`

```
181,88,99,2,14,153,182,192,104,185,57,194,70,190,89,72,46,117,172,183,30,64,40,155,71,60,157,45,134,175,197,132,29,
6,115,97,140,47,21,38,154,170,177,125,26,130,96,121,158,145,28,149,13,146,116,133,136,120,86,135,76,162,54,187,94,1
05,196,114,10,90,20,74,171,39,179,87,82,85,59,51,167,73,151,69,160,184,3,27,11,66,84,80,127,129,52,150,67,24,34,152
```

Instance B, Greedy LS, nodes exchange, greedy start `objective 49913`

```
125,145,28,149,13,146,37,116,147,42,50,191,130,121,198,96,48,156,89,57,104,194,190,70,187,196,114,10,78,54,162,192,
80,127,129,90,153,14,2,20,74,17,171,99,88,179,87,82,51,85,59,73,167,181,152,34,24,67,66,76,135,27,98,151,122,3,184,
46,72,117,8,120,133,136,172,160,69,189,21,183,30,38,154,140,47,64,33,40,141,155,71,60,157,45,134,175,132,29,6,170
```

Instance B, Greedy LS, edges exchange, random start `objective 50579`

```
74,2,14,20,153,182,52,90,129,80,127,10,114,196,105,94,81,187,54,192,162,104,57,89,48,96,198,121,120,133,136,172,8,1
17,160,69,168,189,21,183,30,38,47,140,154,158,149,28,145,65,177,125,170,115,6,103,178,132,29,197,36,175,134,45,157,
60,71,155,40,33,64,85,59,73,167,51,82,87,179,181,152,98,151,122,173,3,184,46,72,86,185,76,135,27,34,24,66,67,99,171
```

Instance B, Greedy LS, edges exchange, greedy start `objective 49727`

```
183,21,38,47,140,154,97,26,125,177,170,115,180,6,29,132,197,36,71,155,141,40,60,157,175,134,45,85,51,59,73,167,82,8
7,179,181,34,24,67,99,171,74,20,2,14,153,90,129,127,80,192,162,54,78,10,114,196,61,105,94,81,169,187,70,190,104,57,
185,76,135,27,3,184,151,69,160,172,136,133,120,72,89,48,96,198,121,42,147,116,37,146,13,149,28,145,158,50,130,191,3
0
```

7. Solution checker

All 16 best solutions above were checked with `checker.py`, which independently re-reads the instance data, verifies that each solution has exactly 100 distinct, in-range node indices, and recomputes length/cost/objective from scratch.

ALL 16 VALID — the recomputed objective matched the value reported by `main.cpp` exactly in every case.

8. Conclusions

- **Two-edges exchange is far better than two-nodes exchange from random starts**: steepest LS reaches 76245.8 vs. 90051.5 on A and 52597.5 vs. 66084.1 on B. A 2-opt move removes a crossing directly. Swapping two nodes changes four edges at once and usually cannot untangle a random tour, so the local optima of the nodes neighborhood are still visibly tangled (section 5). From a greedy start the tour is already untangled, and both intra-route neighborhoods end up at almost the same quality.
- **Starting solution.** On A, LS from greedy starts is clearly best (all four variants ≈73 000; best single solution 71955). It improves the starting construction only slightly (73348.7 → 72979.8), with just ≈3–5 moves per run, because weighted regret already produces solutions close to a local optimum. On B, LS with edges exchange from *random* starts has the best *average* (52597.5) and by far the smallest spread. Greedy starts give the best *single* solution (49346), but they inherit the large spread of the construction (max ≈60 000), which LS with these small neighborhoods cannot repair.
- **Steepest vs. greedy** reach very similar quality. With nodes exchange from random starts, greedy is even somewhat better (87787.9 vs. 90051.5 on A, 63784.8 vs. 66084.1 on B). Its random first-improvement path explores more varied regions than always taking the single best move.
- **Effort.** From random starts, greedy LS evaluates ≈15–20× fewer moves than steepest (≈139 k vs. ≈2.3 M), even though it applies ≈3× more (smaller) improving moves. It runs ≈2× faster (5.3 vs. 10.6 ms, 3.6 vs. 7.6 ms on A). It is not 15× faster because each greedy evaluation needs a random number and a random memory access, while steepest scans the move list sequentially. For greedy starts the LS part is tiny, the run time is dominated by the ≈1.5 ms construction, and this per-move overhead makes greedy LS slightly slower than steepest there.
- Edges-exchange LS needs fewer iterations than nodes-exchange LS (≈130 vs. ≈150 for steepest) and is faster. All local searches stay at ≈2–11 ms per run, so even the slowest variant is cheap at this instance size. The best solutions found so far on both instances come from local search: 71955 on A (previous best 72011) and 49346 on B (previous best 50485).